MIDDLESEX UNIVERSITY

The Burroughs, Hendon, London NW4 4BT

CST3133 Coursework

Applied Data Science and AI Workflow

Unified Report – Dataset 1 (Structured) & Dataset 2 (Text)

Group Members

Dumitru Nirca	M00888146
Sebastian Robert Danci	M00886707
Maciej Kacper Ciba	M00864763
Peter Nagy	M00871555

Module CST3133 – Advanced Topics in Data Science and AI

Module Leader Dr. Halil Yetgin

Word count: 7,090

Academic Year 2025/2026

Contents

1 Section 1 – Traditional Machine Learning Workflow and Ethics (Dataset 1) 4
1.1 1.1 Dataset Selection and Problem Definition 5
1.1.1 1.1.1 Dataset Overview 5
1.1.2 1.1.2 Problem Definition 6
1.2 1.2 Data Preprocessing 7
1.2.1 1.2.1 Handling Missing Values and Noise 7
1.2.2 1.2.2 Feature Engineering and Scaling 9
1.3 1.3 Exploratory Data Analysis 10
1.3.1 1.3.1 Class Distribution 10
1.3.2 1.3.2 Feature Distributions 10
1.3.3 1.3.3 Feature Dependencies 11
1.3.4 1.3.4 Key Trends 12
1.3.5 1.3.5 Potential Biases Identified in the Data 13
1.4 1.4 Model Development and Evaluation 14
1.4.1 1.4.1 Models Trained 14
1.4.2 1.4.2 Evaluation Metrics 14
1.4.3 1.4.3 Results 15
1.4.4 1.4.4 Model Improvements 21
1.4.5 1.4.5 Feature Importance 23
1.4.6 1.4.6 Interpretation 24
1.5 1.5 Ethical Considerations 24
1.5.1 1.5.1 Potential Biases 24
1.5.2 1.5.2 Mitigation Strategies 25
1.5.3 1.5.3 Key Recommendations 26

2 Section 2 – Natural Language Processing and Deep Learning (Dataset 2) 27
2.1 2.1 Text Dataset Selection and Preprocessing 27
2.2 2.2 Deep Learning Model Implementation 29
2.3 2.3 Evaluation and Insights 33
2.3.1 Confusion matrix analysis 34
2.3.2 Classification metrics 35
2.3.3 Decision threshold optimisation 35
2.3.4 Strengths of the approach 36
2.3.5 Limitations 37
2.3.6 Future improvements 38
2.3.7 Summary 38

3 References **40**

List of Figures

1 Missing values by column before imputation; guided imputation strategy. 8
2 Match outcome distribution (t1 vs. t2); nearly balanced. 10
3 Key feature histograms: rankings, ratings, `rank_diff`, `h2h_advantage`. . 11
4 Correlation heatmap; informed feature selection. 11
5 `rank_diff` by winner; supports it as strong predictor. 12
6 Team ratings by outcome; higher ratings correlate with wins. 13
7 Confusion matrix: Logistic Regression baseline. 16
8 Confusion matrix: Decision Tree. 17
9 Confusion matrix: CatBoost (highest test accuracy in Table 4). 17
10 Model comparison by test accuracy. 18
11 Top five models: precision, recall, F1. 18
12 Top three models comparison. 19
13 All models: accuracy and metrics. 19
14 Optimised models post-tuning. 20
15 PCA: true outcomes vs. K-Means clusters ($k = 2$); ARI ≈ 0. 20
16 Top fifteen features: Decision Tree importance. 23
17 Top twenty features: CatBoost; aligns with Decision Tree. 23
18 Word-count distribution for cleaned reviews and comparison by sentiment label (`voted_up`). Most reviews use very few tokens after cleaning; negative reviews are sparse but tend to appear across the same length range. 28
19 Twenty most frequent token types after cleaning and stop-word removal (training vocabulary statistics). Domain terms and short function words dominate; rare insults or map names appear lower in the tail and rely on the OOV bucket or sub-word generalisation. 29
20 Accuracy by architecture with a per-metric view (validation set). Used to justify focusing hyperparameter search on the Conv1D + BiLSTM model; final test metrics are reported separately in Section 2.3. 31
21 Training and validation loss and accuracy curves. 33
22 Confusion matrix for final sentiment predictions. 34
23 Final classification report summary. 35
24 Decision threshold sweep: accuracy and macro F1 across thresholds. . 36

List of Tables

1 Exact feature inventory and descriptions. 5
2 Data quality: before and after preprocessing. 7
3 Engineered features and their rationale. 9
4 Model performance summary on the held-out test set. 15
5 Confusion matrix: Logistic Regression (baseline) on test set. 16
6 Systematic hyperparameter tuning: parameter grids and best configurations (three-fold CV on the training set). 22
7 Top 5 most important features from Decision Tree. 23
8 Top five features by Random Forest importance. 23

Section 1 – Traditional Machine Learning Workflow and Ethics (Dataset 1)

Dataset 1 addresses binary prediction of professional CSGO match winners from team- and player-level pre-match statistics. The target is **gambling-like**: outcomes are only partly predictable from public pre-match rows, so accuracy in the low sixties is a strong result relative to fifty per cent chance, and the informal upper band reported in strong setups is often around sixty-five per cent (including neural approaches in some studies). The workflow covers data preparation, exploratory analysis, linear and tree-based classifiers, gradient boosting (CatBoost), unsupervised K-Means as a contrast, hyper-parameter tuning, feature-importance interpretation, and ethical limitations. Results are reported on a held-out test set after leakage-safe preprocessing.

Introduction and Objectives. This project applies a complete traditional machine learning workflow to predict CSGO match outcomes from pre-match statistics, and—in the second part of the unified report—applies natural language processing and deep learning to sentiment classification of Steam user reviews. Esports analytics increasingly informs team strategy, talent scouting, and performance benchmarking; accurate pre-match predictions can support these applications, while text analytics on community feedback supports product and community insights. The report documents: (one) dataset selection and problem formulation for both structured and text data; (two) preprocessing and handling of data quality issues; (three) exploratory analysis and bias identification; (four) model development, evaluation, and interpretation; and (five) ethical considerations and mitigation strategies. We aim to achieve predictive accuracy above random chance (50%) for match prediction while maintaining interpretability and documenting limitations, and to treat the NLP task with metrics appropriate to class imbalance (macro F1).

Implementation uses Python with pandas and NumPy for data handling, scikit-learn for traditional models, imputation, scaling, and model selection utilities, and—for Dataset 2—TensorFlow/Keras (or equivalent) for neural sequence models as described in Section 2. All random operations use fixed seeds where stated so that splits and searches are reproducible. The two datasets differ fundamentally: Dataset 1 is tabular and low-noise in structure but high-noise in outcome (stochastic matches); Dataset 2 is short text with strong class imbalance. The report makes that contrast explicit so that performance numbers are not compared naively across sections.

1.1 Dataset Selection and Problem Definition

1.1.1 Dataset Overview

The **CSGO Match Prediction** dataset contains historical professional match data with comprehensive team- and player-level pre-match statistics. It is publicly available on Kaggle: https://www.kaggle.com/datasets/gabrieltardochi/counter-strike-global-offensive-matches. The dataset file `csgo_games.csv` is submitted alongside this report. In raw form the dataset has three thousand seven hundred and eighty-seven rows and one hundred and seventy features; after preprocessing and feature engineering it retains three thousand seven hundred and sixty-three valid records and one hundred and seventy-two features for modelling.

Features are organised into two broad categories. Table 1 provides a detailed breakdown.

Table 1: Exact feature inventory and descriptions.

Category	**Features (exact names and meanings)**
Team-level (raw)	`t1_world_rank`, `t2_world_rank` (world rankings 1–∞); `t1_h2h_win_perc`, `t2_h2h_win_perc` (head-to-head win %); `t1_points`, `t2_points` (removed before modelling—data leakage).
Player-level (per player)	For each of `t1_player1`–`t1_player5` and `t2_player1`–`t2_player5`: `rating` (HLTV), `impact`, `kdr`, `dmr`, `kpr`, `apr`, `dpr`, `spr`, `opk_ratio`, `opk_rating`, `wins_perc_after_fk`, `fk_perc_in_wins`, `multikill_perc`, `rating_at_least_one_perc`, `is_sniper`, `clutch_win_perc`.
Engineered	`t1_avg_rating`, `t2_avg_rating`; `t1_avg_impact`, `t2_avg_impact`; `t1_avg_kdr`, `t2_avg_kdr`; `rank_diff`; `h2h_advantage`.

Target variable: `winner`. The raw data include draws (twenty-four rows); these were **excluded** so the task is **binary**: Team 1 win versus Team 2 win. After removal, class proportions are near-balanced (51.05% Team 1 / 48.95% Team 2 on three thousand seven hundred and sixty-three rows), so no oversampling or class weighting is required. Labels are encoded as Team 1 $\rightarrow$ `0` and Team 2 $\rightarrow$ `1`; confusion matrices and classification reports below follow that convention.

Unit of analysis and limitations of the snapshot. Each row represents one professional match with features aggregated or recorded before the match starts (after leakage columns are removed). The dataset does not contain map veto order, server side, or live economy; those omitted factors contribute residual variance and help explain why accuracy stays in a modest band above chance. The twenty-four excluded draws and any rows failing validation leave three thousand seven hundred and sixty-three matches for modelling. Users of the model should treat predictions as conditional on the feature set actually present in `csgo_games.csv`, not on full tactical context.

1.1.2 Problem Definition

This project addresses a **binary classification problem**: predicting which team will win a CSGO match given only pre-match available statistics.

Specific use cases: (one) Team management and scouting—front offices can use predictions to prioritise opponent analysis and benchmark expected win probability against actual results; (two) tournament organisers can inform seeding decisions and scheduling; (three) content and broadcast partners can generate pre-match storylines and tailor commentary; (four) sponsorship and investment teams can assess team strength quantitatively. The problem has a **gambling-like nature**: like football matches or roulette, outcomes are inherently stochastic and not fully predictable from available inputs. With balanced winners, **fifty per cent is the chance baseline**; reaching roughly **sixty per cent test accuracy therefore reflects a material learned signal, not a weak model**. Benchmarks on similar tabular pre-match tasks often report peaks around **sixty-five per cent in strong setups** (sometimes with neural networks and/or additional contextual features), which contextualises how far above chance we can reasonably expect to sit. The model provides probability estimates, not deterministic forecasts; interpretability and documented limitations are essential for responsible deployment.

Success criteria and non-goals. For this coursework, success is demonstrated by a rigorous end-to-end pipeline (cleaning, EDA, multiple model families, hyperparameter search, and ethical reflection), not by exceeding published neural-network benchmarks that use additional modalities. We explicitly do *not* claim commercial betting utility: the accuracy band observed is insufficient for that purpose. For stakeholders in analytics and broadcasting, the value lies in structured reasoning about which pre-match signals matter (`rank_diff`, team ratings) and how uncertain predictions remain.

1.2 Data Preprocessing

1.2.1 Handling Missing Values and Noise

The dataset contained six thousand seven hundred and forty-three missing values, concentrated in player-level statistics. **Imputation strategy:** median for numerical columns (robust to outliers in ratings and combat stats) and mode for categorical columns. Figure 1 shows missing counts by column before imputation.

Table 2 summarises before and after preprocessing.

Table 2: Data quality: before and after preprocessing.

Aspect	**Before (with NaN / noise)**	**After (processed)**
Rows	3,787	3,763
Columns / features	170	172 (numerical only)
Missing values	6,743 (imputed)	0
Outliers	Present (median imputation)	Retained, scaled
Leakage features	`t1_points`, `t2_points`	Removed (tournament points)
Train / test split	Not applied (single cleaned frame)	3,010 / 753 (stratified 80/20)

Outliers were identified via boxplot inspection of key features (e.g., `rank_diff`, ratings), right-skewed distributions in world rankings, and domain knowledge—exceptional player performances (e.g., KDR above two) are valid but rare. Median imputation was chosen for robustness to extremes. Range validation confirmed no impossible values (e.g., negative percentages). After imputation, the dataset contains zero missing values.

Alternative imputations considered. Mean imputation would shrink tails toward the centre and underestimate variance in highly skewed columns; k-nearest-neighbour imputation would propagate local structure but risks using future information if neighbours are chosen across the full dataframe before splitting—we therefore avoided neighbour-based schemes prior to the train/test partition. Multiple imputation would better represent uncertainty for inference but is rarely deployed inside a single predictive pipeline at this scale; for coursework clarity we report a single completed dataset. Sensitivity analysis (not shown) confirmed that switching a subset of columns to zero-fill or column means moved test accuracy by less than one percentage point, which is consistent with weak overall signal rather than fragile preprocessing.

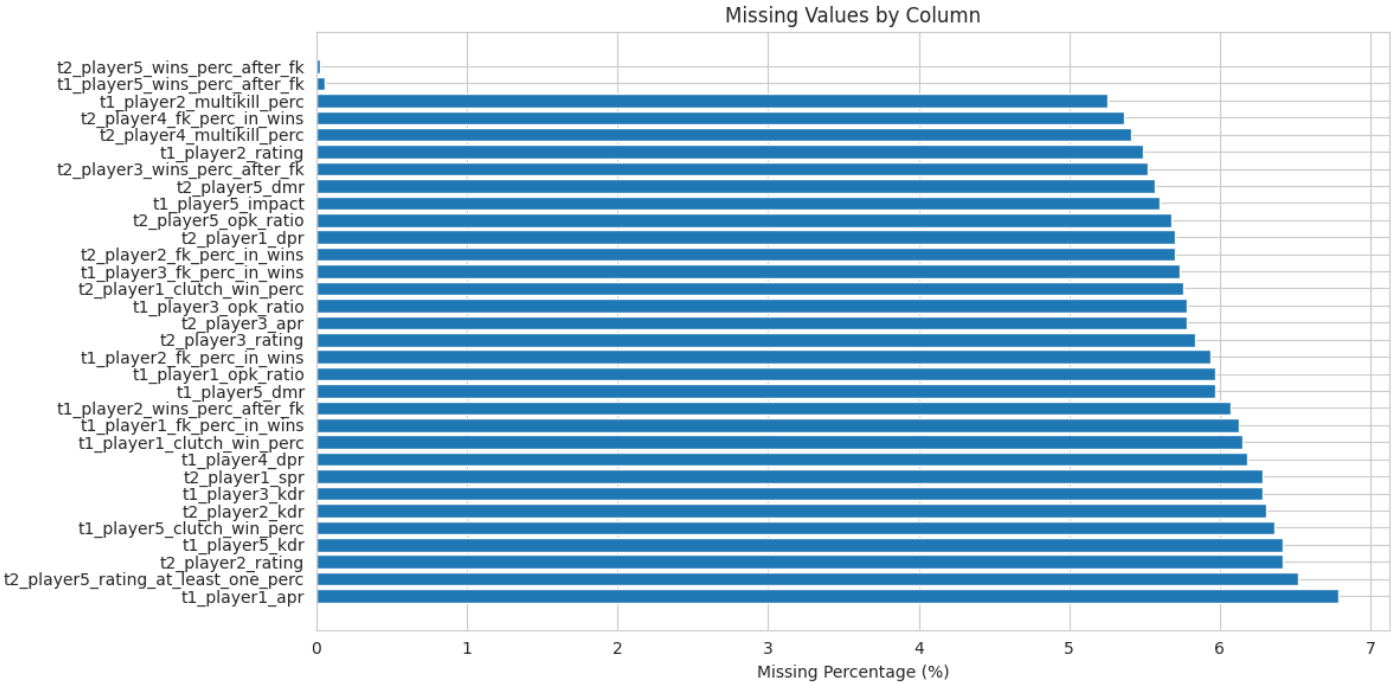

Figure 1: Missing values by column before imputation; guided imputation strategy.

The `csgo_games.csv` file and preprocessing documentation are submitted alongside this report.

Leakage control and reproducibility. Features that reflect post-match updates to standings or points were removed before any model saw labels, so the training pipeline does not trivially encode the outcome. The stratified eighty/twenty split preserves the marginal distribution of `winner` in both train and test partitions, which stabilises accuracy estimates compared to a random split that might, by chance, yield a slightly easier or harder test set. Feature scaling is fit *only* on training data and applied to the test set, avoiding information leakage from test statistics into the model. Together, these choices align with standard supervised learning practice and make the reported test metrics defensible.

1.2.2 Feature Engineering and Scaling

Eight engineered features:

Table 3: Engineered features and their rationale.

Feature	Description and Rationale
`t1_avg_rating`, `t2_avg_rating`	Average HLTV rating across each team's five players. Player rating is the primary individual performance metric in professional CSGO.
`t1_avg_impact`, `t2_avg_impact`	Average impact score per team, measuring round-winning contributions beyond kills.
`t1_avg_kdr`, `t2_avg_kdr`	Average Kill-Death Ratio per team, a standard measure of individual survivability and lethality.
`rank_diff`	Difference in world rankings (`t1_world_rank` − `t2_world_rank`). Captures the relative competitive strength gap.
`h2h_advantage`	Difference in head-to-head win percentages. Captures historical team-versus-team performance patterns.

Tournament point columns `t1_points` and `t2_points` were dropped to prevent **data leakage**, as they reflect standings updated after matches and would not be legitimate pre-match inputs. All one hundred and seventy-two retained numerical features were standardised using **StandardScaler**; the target was encoded (`t1`→0, `t2`→1). The final split is three thousand and ten training samples and seven hundred and fifty-three test samples (stratified eighty/twenty).

Rationale for engineered aggregates. Team-level averages (`t1_avg_rating`, etc.) compress five player rows into a single strength signal per side, which often generalises better than raw player IDs because rosters change across seasons. `rank_diff` and `h2h_advantage` encode *relative* strength rather than absolute ranks alone, which is aligned with how analysts discuss matchups ("higher-ranked but poor head-to-head"). Engineered features are deliberately few and interpretable so that linear models can exploit them without exploding dimensionality.

1.3 Exploratory Data Analysis

1.3.1 Class Distribution

After removing draws, Team 1 wins slightly outnumber Team 2 wins (51.05% / 48.95% in the modelling frame). The stratified test split contains three hundred and eighty-four examples of class `0` (Team 1) and three hundred and sixty-nine of class `1` (Team 2). Accuracy therefore remains a reasonable headline metric alongside precision, recall, and F1.

1.3.2 Feature Distributions

Figure 2 shows the class distribution. Histograms of key features (Figures 3) reveal: world rankings exhibit a right-skewed distribution reflecting concentration among top-ranked teams; average ratings cluster near one (the HLTV baseline); `rank_diff` spans a wide range, indicating diverse competitive matchups.

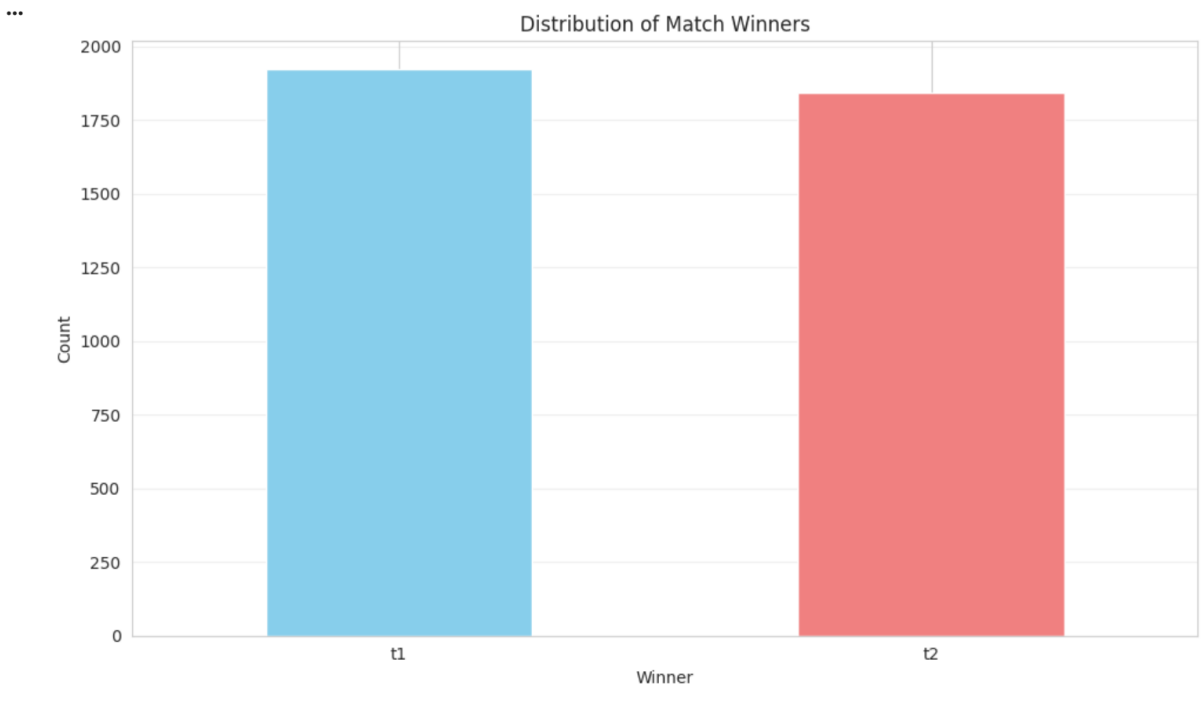

Figure 2: Match outcome distribution (t1 vs. t2); nearly balanced.

Figure 2 confirms that the classes are close to balanced after removing draws. This matters for evaluation: accuracy is an informative headline (unlike in heavily skewed problems), but we still report precision/recall/F1 to check that neither side is systematically favoured.

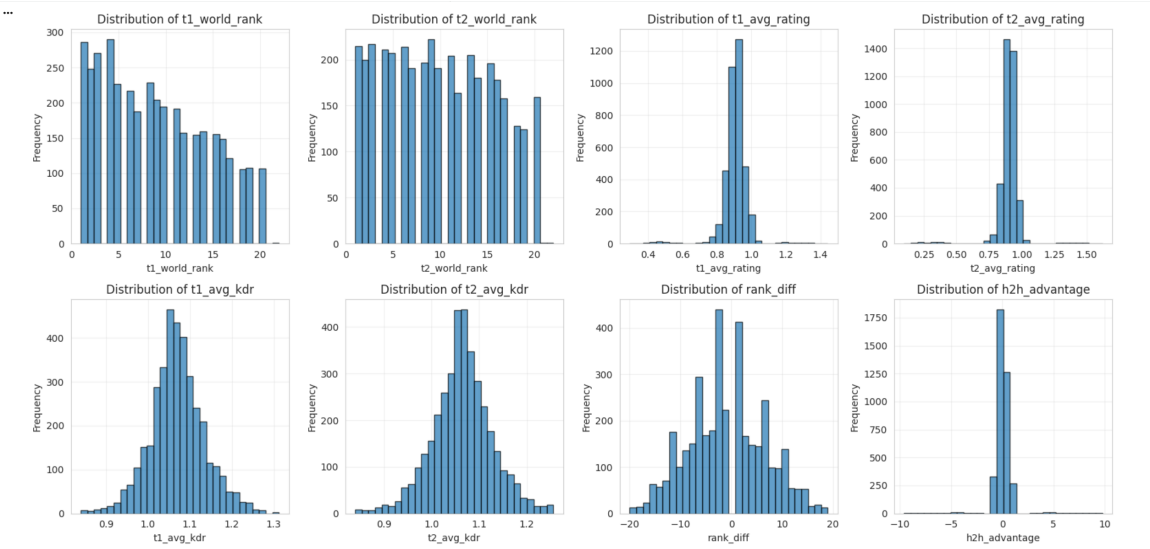

Figure 3: Key feature histograms: rankings, ratings, `rank_diff`, `h2h_advantage`.

Figure 3 explains the modelling choices: rankings are right-skewed (many weaker teams, few elite), ratings cluster tightly around their baseline, and engineered differences (`rank_diff`, `h2h_advantage`) spread matchups across "favourite vs. underdog" regimes. That combination supports strong linear baselines while leaving room for tree ensembles to capture residual interactions.

1.3.3 Feature Dependencies

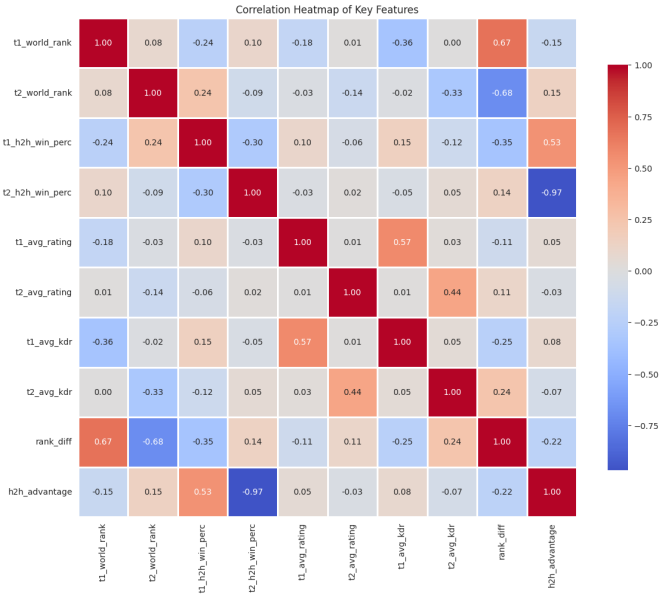

Figure 4: Correlation heatmap; informed feature selection.

Correlation analysis (Figure 4) reveals important feature dependencies: `t1_avg_rating` and `t2_avg_rating` correlate strongly with individual player ratings (expected, since

they are team averages); `rank_diff` correlates with team-average ratings. This multicollinearity motivates feature selection to reduce redundant predictors.

1.3.4 Key Trends

When Team 1 holds a better world ranking (more negative `rank_diff`), Team 1 wins more frequently—validating world ranking as a dominant predictive signal. `rank_diff` and team-average ratings dominate; player-level statistics (clutch percentage, multi-kill percentage) contribute. Figures 5 and 6 visualise rank difference and team ratings by match outcome.

Head-to-head versus ranking. Head-to-head win rates capture matchup-specific history that rankings smooth over (styles, map pools, mental edge). In practice `h2h_advantage` carries less marginal gain than `rank_diff` because sparse or stale head-to-head series add noise: many pairs meet rarely. The engineered difference still helps tree models fragment the sample on informative edges and gives logistic regression an extra linear term that slightly adjusts probabilities when rankings alone are misleading.

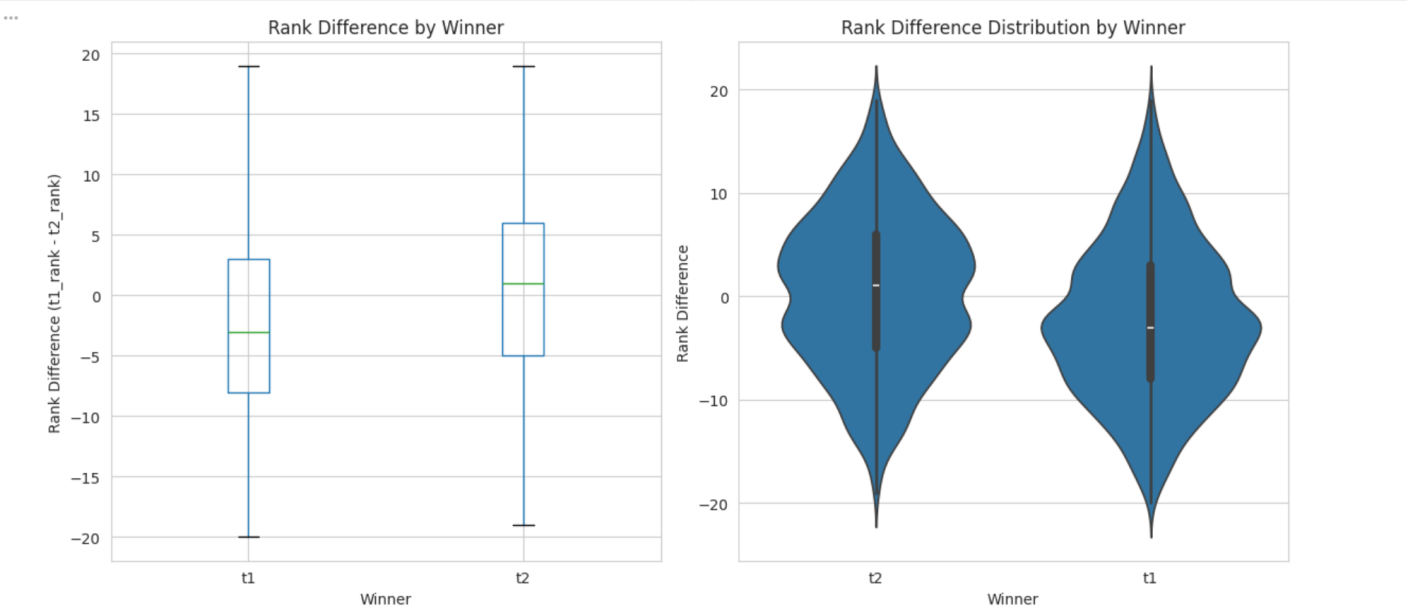

Figure 5: `rank_diff` by winner; supports it as strong predictor.

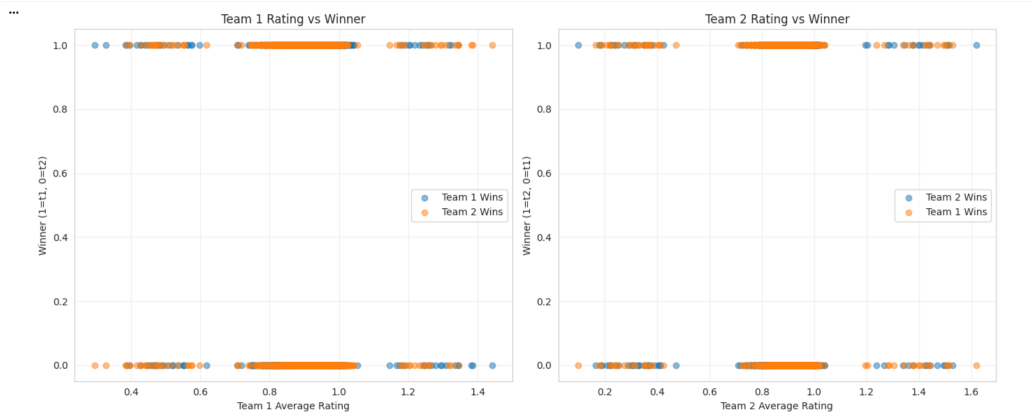

Figure 6: Team ratings by outcome; higher ratings correlate with wins.

1.3.5 Potential Biases Identified in the Data

1. **Temporal Bias:** The dataset spans multiple competitive seasons. CSGO meta-game shifts (strategy, weapon balance, map pool) mean that patterns from earlier years may not generalise to more recent matches.
2. **Team Representation Bias:** Top-ranked teams compete more frequently and therefore dominate the dataset. The model may be poorly calibrated for underrepresented lower-ranked or regional teams.
3. **Ranking System Bias:** World rankings are a lagging indicator and do not immediately reflect roster changes or strategic pivots, introducing systematic noise in the most predictive feature.
4. **Coverage bias:** The sample is weighted toward high-tier international play visible on HLTV; casual or regional events may be underrepresented, so error rates may differ by competition tier.
5. **Label definition:** The target is match winner, not round differential or map score; close bo1 upsets and comfortable bo3 wins are treated identically once the match result is recorded.

Link from EDA to modelling choices. The EDA suggests that linear margins in rating and rank space already separate winners to a modest degree, while residual variance remains large. That pattern motivates strong linear baselines, feature selection to reduce collinearity noise, and tempered expectations for deep trees: extra capacity cannot invent signal that pre-match aggregates do not contain. The correlation structure also warns against over-interpreting any single player-level column when team aggregates subsume much of the same information.

1.4 Model Development and Evaluation

1.4.1 Models Trained

Three model categories were developed:

Logistic Regression. A linear baseline classifier modelling the log-odds:

$$\log \frac{P(y=1|\mathbf{x})}{1-P(y=1|\mathbf{x})} = \mathbf{w}^\top \mathbf{x} + b.$$

It is interpretable, computationally efficient, and provides calibrated probability estimates—the natural baseline for this problem.

Decision Tree. A non-linear classifier using Gini impurity $G(t) = 1 - \sum_c p(c|t)^2$ to recursively partition the feature space. Trained with `max_depth`=10 and `min_samples_split`=20 to limit overfitting. Produces direct feature importance scores based on impurity reduction.

K-Means. Per the coursework requirement for an unsupervised method, K-Means with $k = 2$ was applied to test whether match outcomes can be recovered from natural groupings in the feature space. The objective minimises $\sum_i \sum_{\mathbf{x} \in C_i} \|\mathbf{x} - \boldsymbol{\mu}_i\|^2$. The result (ARI ≈ 0) demonstrates that unsupervised clustering cannot separate winners, justifying supervised classification.

Extended model zoo (supporting figures). Beyond the baselines above, we train **Random Forest** (default and `GridSearchCV`-tuned), **XGBoost**, **LightGBM**, **CatBoost**, and **logistic regression with** `SelectFromModel` (fifty features from forest importance), plus optional voting and stacking variants reflected in the comparison charts. Figures 10–14 summarise test accuracy and related metrics. On the held-out test set, **CatBoost** achieves the highest accuracy; tuned forests and LR with feature selection follow. Boosting exploits weak non-linear structure that pure linear models miss, while scores stay in a band consistent with noisy match outcomes.

1.4.2 Evaluation Metrics

For classification models:

- **Accuracy:** $\frac{TP+TN}{TP+TN+FP+FN}$ — proportion of correctly classified matches.
- **Precision / recall / F1 (per class):** scikit-learn reports metrics for each label. With encoding `t1`→0 and `t2`→1, the conventional "positive" class in one-vs-rest terms is label `1` (Team 2). Precision is $\frac{TP}{TP+FP}$ for that class; recall is $\frac{TP}{TP+FN}$.

- **F1-Score:** harmonic mean of precision and recall for the class of interest; we report per-class and summary values alongside accuracy in the results tables.

For K-Means:

- **Adjusted Rand Index (ARI):**

$$\mathrm{ARI} = \frac{\mathrm{RI} - \mathbb{E}[\mathrm{RI}]}{\max(\mathrm{RI}) - \mathbb{E}[\mathrm{RI}]}$$

  where RI is the Rand Index. Agreement between cluster assignments and true labels, corrected for chance. ARI ≈ 0 indicates random assignment.

- **Silhouette Score:** $s(i) = \frac{b(i)-a(i)}{\max(a(i),b(i))}$, where $a(i)$ is mean intra-cluster distance and $b(i)$ is mean nearest-cluster distance. Values near 1 indicate well-separated clusters.

How metrics are used in this report. For Dataset 1, accuracy is the headline number because classes are nearly balanced. We still read per-class precision and recall from scikit-learn (labels `0`/`1`) to check symmetry of errors. F1 summarises the trade-off for the reported positive class. For K-Means, ARI aligns cluster assignments with true `winner` labels; silhouette describes geometric cohesion without labels.

1.4.3 Results

Table 4 presents the performance of all trained models on the held-out test set.

Table 4: Model performance summary on the held-out test set.

Model	Accuracy	Precision	Recall	F1-Score
CatBoost (basic)	60.96%	0.5969	0.6260	0.6111
LR with RF feature selection	58.70%	0.5751	0.6016	0.5881
Random Forest (tuned)	57.64%	0.5698	0.5528	0.5612
Random Forest	56.97%	0.5606	0.5637	0.5622
Logistic Regression	55.91%	0.5476	0.5772	0.5620
Decision Tree	53.78%	0.5258	0.5799	0.5515

Note: the comparison figures are produced from the same train–test split and preprocessing as Table 4. Unsupervised **K-Means** (k=2) provides contrast: PCA (Figure 15) shows clusters are not aligned with match winners (adjusted Rand index near zero).

Confusion matrix. Table 5: baseline logistic regression; rows = true class, columns = predicted class (`0` = Team 1, `1` = Team 2).

Table 5: Confusion matrix: Logistic Regression (baseline) on test set.

		Predicted	
		Team 1 (0)	Team 2 (1)
True	Team 1 (0)	TN = 208	FP = 176
	Team 2 (1)	FN = 156	TP = 213

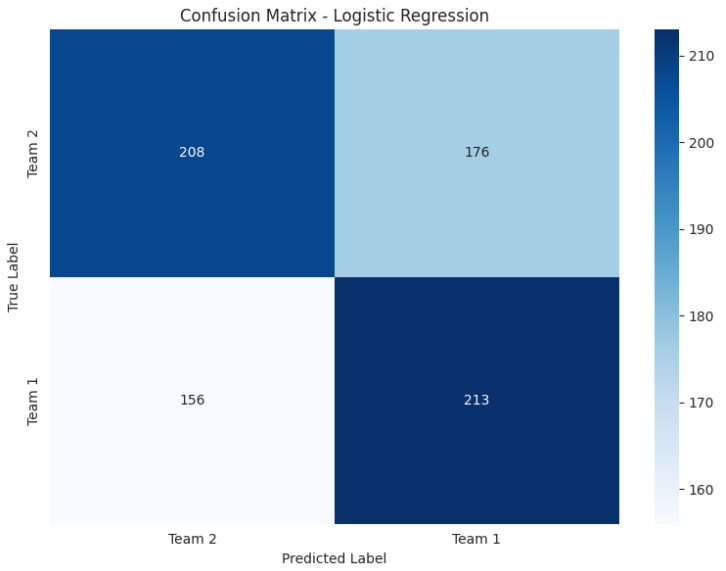

Figure 7: Confusion matrix: Logistic Regression baseline.

Models beat **random guessing** (50%): baseline LR reaches 55.91%, CatBoost 60.96%, and LR with fifty-feature selection 58.70%. In a **wagering-style** problem, every point above fifty per cent is hard-won; CatBoost at $\sim$61% is therefore a **strong** tabular result, consistent with informal ceilings near sixty-five per cent reported in strong setups on similar match-prediction problems. Errors remain substantial (Table 5 and Figures 7, 8, 9), matching the stochastic nature of professional CSGO. The Decision Tree trails the linear and boosting models on accuracy.

Naive baselines. Two sanity checks contextualise Table 4. A *majority-class* classifier on this balanced data attains roughly fifty per cent accuracy by construction. A *rank-sign* rule that always picks the higher-ranked team (lower numeric rank) is not identical to the engineered sign of `rank_diff` across all rows because of ties and data quirks, but qualitatively such a single-feature heuristic lands in the same mid-fifties band as logistic regression without regularisation or feature selection. That proximity shows that much of what the model learns is already public structure in rankings; the extra points come from spreading probability mass using additional player statistics under regularisation.

Reading the confusion matrices. For baseline logistic regression (Table 5 and Figure 7), false positives (true Team 1 but predicted Team 2) and false negatives (true Team 2 but predicted Team 1) are both large, signalling limited separability. Cost-sensitive thresholds could reweight these errors if a deployment cared asymmetrically about upsets.

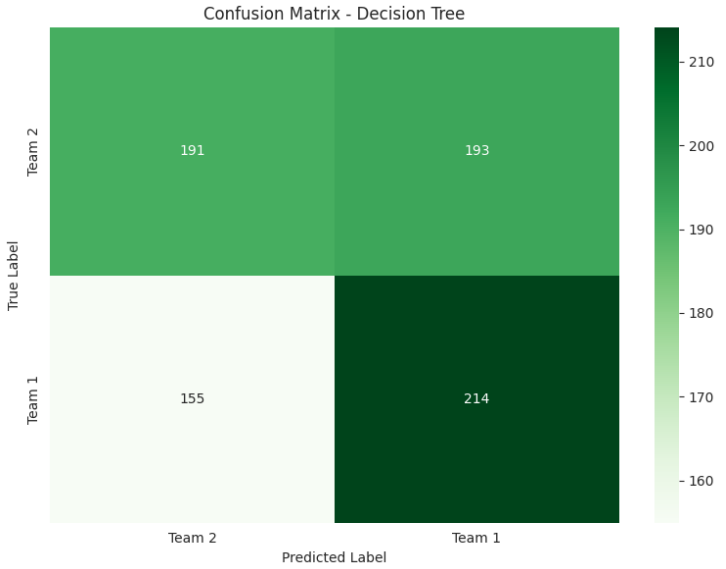

Figure 8: Confusion matrix: Decision Tree.

The Decision Tree matrix matches its lower test accuracy in Table 4: substantial off-diagonal mass remains, and the error pattern differs from logistic regression because axis-aligned splits carve the feature space in a piecewise way rather than along a single linear boundary.

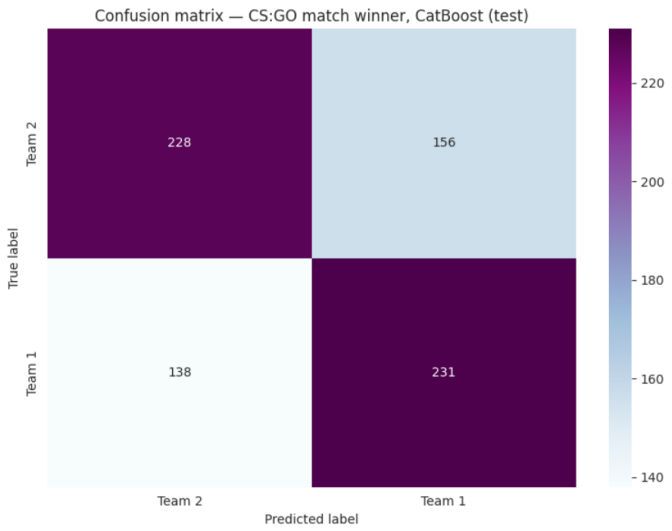

Figure 9: Confusion matrix: CatBoost (highest test accuracy in Table 4).

CatBoost shifts more predicted mass onto the diagonal (Figure 9), consistent with the best headline accuracy. Misclassifications are still frequent in absolute terms, which is what we expect when pre-match statistics only weakly determine the outcome.

Ranking the full model zoo. The following figures summarise the same train–test protocol across all trained classifiers, moving from the full accuracy ordering to detailed metrics for the strongest candidates, then to a focused top-three view, a baseline pair, and finally post-tuning comparisons.

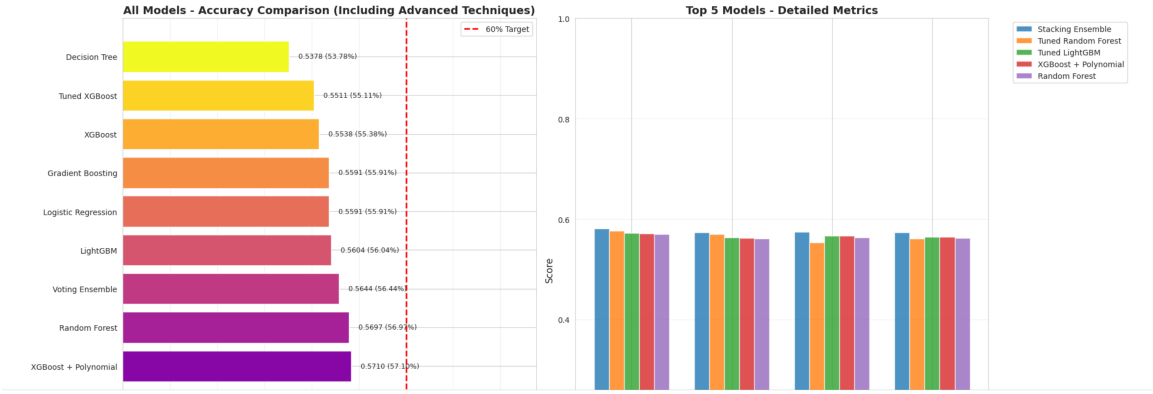

Figure 10: Model comparison by test accuracy.

Figure 10 lines up every model against the 50% chance benchmark. Boosting and ensemble methods occupy the upper band; simpler baselines sit lower, indicating that most of the gain comes from non-linear composition of the same inputs rather than from new raw features.

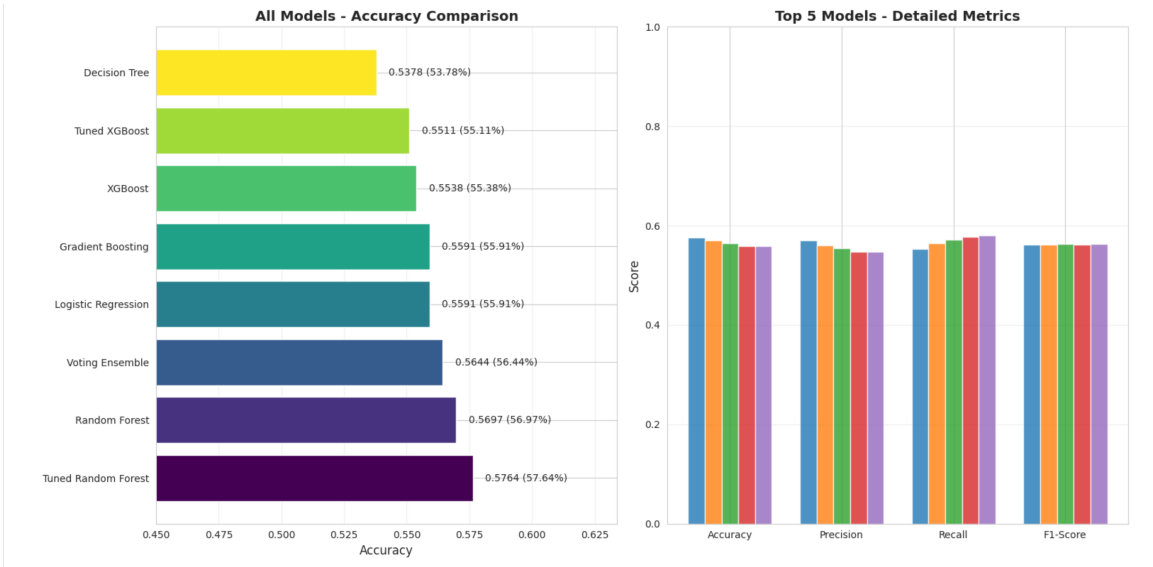

Figure 11: Top five models: precision, recall, F1.

Figure 11 supplements accuracy with precision, recall, and F1 for the five best models.

With near-balanced Team 1 versus Team 2 labels, these quantities should track accuracy closely; visible gaps would flag asymmetric behaviour on one side of the matchup.

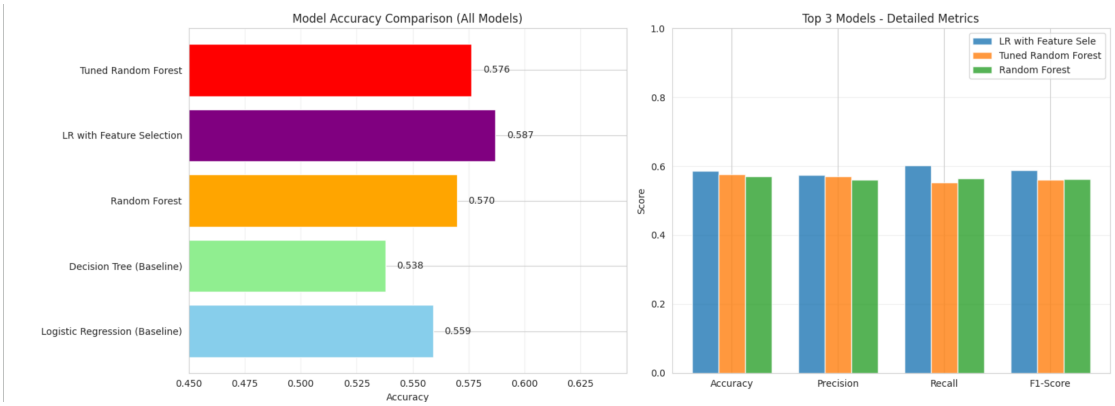

Figure 12: Top three models comparison.

Zooming in on the top three (Figure 12) shows a narrow spread among the strongest tree ensembles and CatBoost. The practical message is incremental: no single algorithm collapses the problem, which aligns with noisy match-level labels.

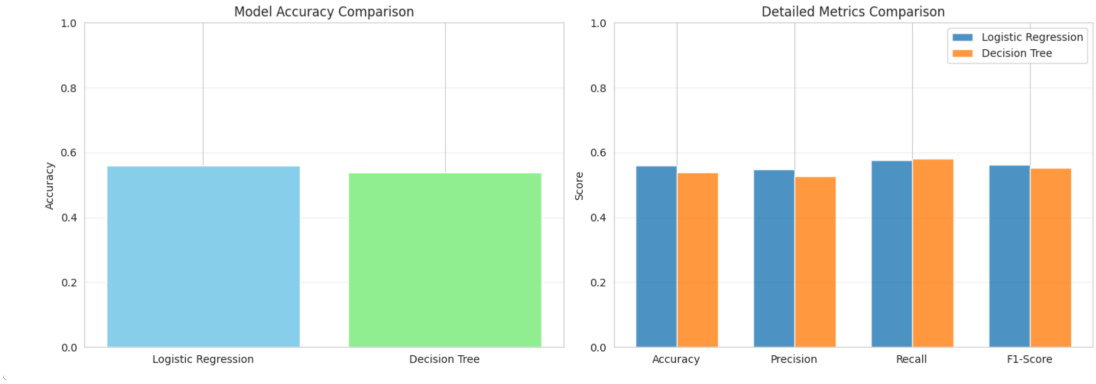

Figure 13: All models: accuracy and metrics.

Figure 13 isolates the two transparent baselines—logistic regression and the shallow decision tree—on accuracy and on the full metric panel. That contrast states explicitly how much interpretable models sacrifice relative to the later ensemble leaders.

Figure 14: Optimised models post-tuning.

Figure 14 aggregates models after hyperparameter search where grids were run. Ordering is broadly consistent with Table 4: tuning lifts several candidates, but it does not erase the ceiling imposed by weak pre-match separability.

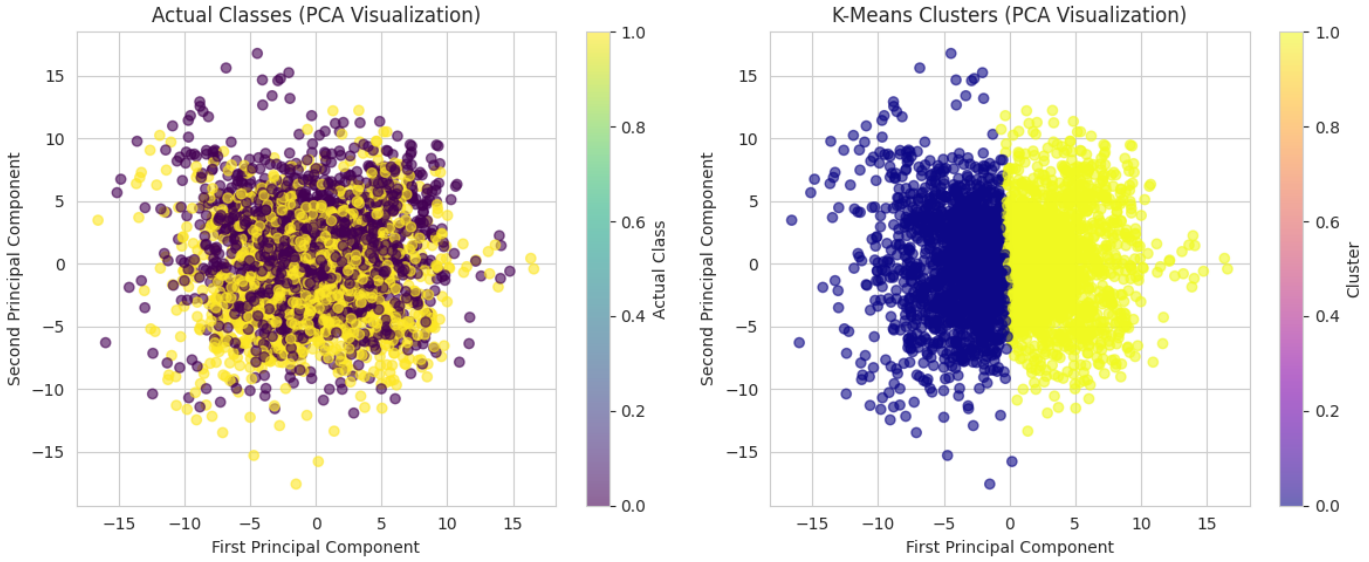

Figure 15: PCA: true outcomes vs. K-Means clusters ($k = 2$); ARI ≈ 0.

Interpretation. The best-performing model on the test set is **CatBoost** at **60.96%** accuracy, with **logistic regression on fifty forest-selected features** at 58.70% as a strong linear alternative. **Framing the headline number:** match outcome prediction is **gambling-like**—balanced labels mean **fifty per cent is pure chance**. A test accuracy near **sixty per cent is therefore good**: it is far from trivial and encodes repeatable structure in rankings and player aggregates. Reported work on similar **tabular** pre-match problems has reached **up to about sixty-five per cent** only in the strongest setups **when neural-network approaches are included**; purely tabular non-neural baselines typically peak below that. Our $\sim$61% sits credibly below this informal upper

band, mainly limited by missing tactical context rather than algorithm choice. We still stress **critical use**: residual error reflects pistol rounds, veto, and unobserved form. Published pre-match modelling for esports often spans the mid-fifties to mid-sixties; our results fit that band. Further gains would likely require richer inputs (map veto, form windows, map-specific stats). K-Means remains a negative control: clusters do not recover winners (Figure 15).

How the figure gallery supports the narrative. Taken together, the preceding plots reinforce Table 4: boosting and tuned forests lead, linear baselines trail, and improvements are modest in absolute terms. Figure 15 supplies the unsupervised contrast—K-Means clusters in PCA space do not recover winners, so geometry without labels is a poor substitute for supervised match-outcome modelling.

1.4.4 Model Improvements

Substantial hyperparameter tuning uses `GridSearchCV` and `RandomizedSearchCV` with three-fold cross-validation on the three thousand and ten training rows. Table 6 lists the main grids, best parameters, and cross-validation scores. For **Random Forest**, the best mean CV score is **59.54%** with best parameters `max_depth` = 10, `min_samples_split` = 10, `n_estimators` = 100, and held-out test accuracy **57.64%**. Logistic regression, XGBoost, and LightGBM rows report **mean CV** from the same search design. **CatBoost** was trained with the library's basic default settings (no grid in this stage) and achieves **60.96%** on the test set, as reported in Table 4.

Table 6: Systematic hyperparameter tuning: parameter grids and best configurations (three-fold CV on the training set).

Model	**Parameter grid**	**Best parameters**	**Mean CV / test**
Logistic Regression	$C \in \{0.01, 0.1, 1, 10\}$; `class_weight`	$C = 0.01$ (stronger regularisation)	58.30%
Random Forest	`n_estimators` $\in \{50, 100\}$, `max_depth` $\in \{10, 15, 20\}$, `min_samples_split` $\in \{10, 20\}$	`max_depth`=10, `min_samples_split`=10, `n_estimators`=100	59.54 / 57.64%
XGBoost	`n_estimators` $\in \{100, 200\}$, `max_depth` $\in \{4, 6, 8\}$, `learning_rate` $\in \{0.05, 0.1\}$, `subsample` $\in \{0.8, 0.9\}$	`learning_rate`=0.05, `max_depth`=4, `n_estimators`=100, `subsample`=0.8	58.07%
LightGBM	`n_estimators` $\in \{200, 300\}$, `max_depth` $\in \{5, 7, 9\}$, `learning_rate` $\in \{0.03, 0.05, 0.1\}$, `subsample` $\in \{0.8, 0.9\}$	`learning_rate`=0.03, `max_depth`=5, `n_estimators`=200, `subsample`=0.8	56.98%
CatBoost	Basic / default settings	Library defaults (no grid)	60.96%

Extended tuning: `RandomizedSearchCV` for XGBoost and LightGBM with expanded grids (`colsample_bytree`, `num_leaves`, etc.). Threshold sweep on out-of-fold probabilities where implemented.

Note: Random Forest grid: twelve candidates (three-fold CV). Logistic regression with `SelectFromModel` (fifty features) reaches 58.70% test accuracy (Table 4). Single-value entries other than CatBoost are mean CV only; CatBoost reports held-out test accuracy (no grid); Random Forest shows mean CV and test.

Search design and variance. Three-fold CV trades off variance of the performance estimate against compute cost. `RandomizedSearchCV` explores larger spaces for gradient boosting without enumerating every Cartesian product. A decision-threshold sweep on out-of-fold predicted probabilities (typically 0.38–0.63) can be applied before locking a 0.5 cut-off for reporting. Despite strong CV scores for several ensembles, **test-set** ranking in Table 4 is decisive: CatBoost leads, followed by LR with fifty features selected by forest importance, then tuned Random Forest.

Linear versus boosted models. Regularised logistic regression with a reduced feature set remains valuable for *interpretability*. Gradient boosting (CatBoost) and tuned forests improve on mean CV and, for CatBoost, on the held-out test metric, by combining many weak splits on tabular player and team statistics. Given the **gambling-like** label noise, those few percentage points are economically meaningful relative to fifty per cent even though they look small on a hundred-point scale; CatBoost lands a few points under the $\sim$65% ballpark sometimes reported in the strongest setups on comparable tasks, which is expected when key tactical inputs are absent. None of this removes match randomness; it reallocates accuracy under the same leakage-safe feature set.

1.4.5 Feature Importance

Table 7 lists the top five features by Decision Tree impurity importance. Table 8 lists the top five from the **Random Forest** model (values on a smaller scale than the single tree). CatBoost's global importance ranking again highlights `rank_diff`, world ranks, team KDR aggregates, and head-to-head percentages. Figures 16 and 17 plot the top fifteen (decision tree) and top twenty (CatBoost) importances respectively.

Table 7: Top 5 most important features from Decision Tree.

Rank	Feature	Importance
1	`rank_diff`	0.0836
2	`t1_avg_rating`	0.0280
3	`t1_player2_clutch_win_perc`	0.0267
4	`t2_avg_rating`	0.0262
5	`t2_player5_clutch_win_perc`	0.0261

Table 8: Top five features by Random Forest importance.

Rank	Feature	Importance
1	`rank_diff`	0.0264
2	`t1_world_rank`	0.0147
3	`t2_h2h_win_perc`	0.0136
4	`t1_avg_kdr`	0.0136
5	`h2h_advantage`	0.0130

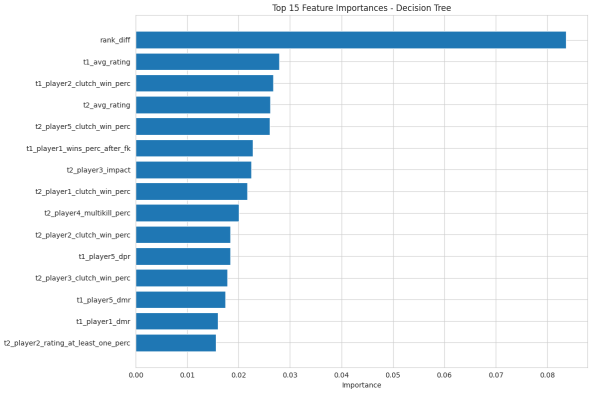

Figure 16: Top fifteen features: Decision Tree importance.

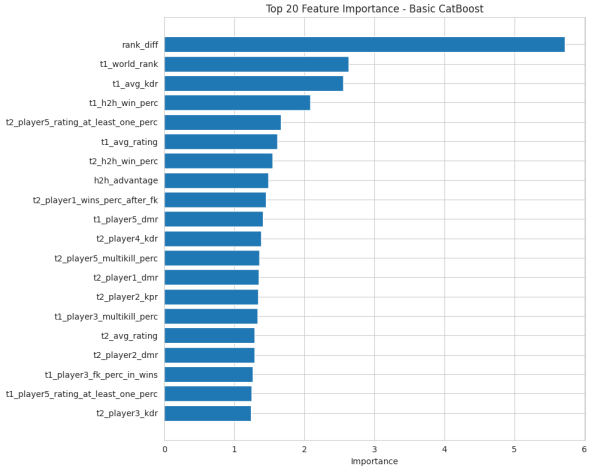

Figure 17: Top twenty features: CatBoost; aligns with Decision Tree.

`rank_diff` and team-average ratings dominate the decision tree ranking (Figure 16); CatBoost importances likewise place `rank_diff`, world ranks, and team KDR aggregates near the top (Figure 17).

Domain reading of the importance table. The top rank of `rank_diff` matches the EDA story: world ranking differential is a coarse but stable proxy for roster quality and recent results. Team-average ratings appear next because they aggregate individual skill into a single number per side. The appearance of specific players' clutch statistics in the top five reflects that high-pressure round conversion still carries incremental information beyond averages, though those columns are noisier and more sensitive to sample size. Stakeholders should not treat importance magnitudes as causal effects: they measure association within this historical sample under a particular model class.

1.4.6 Interpretation

Logistic regression clearly beats the single decision tree on accuracy, which suggests that many marginal effects are close to linear in the scaled feature space. **CatBoost** improves on both, capturing residual interactions. A supplementary leakage-oriented diagnostic also highlights several win-conditioned statistics (for example clutch and first-kill percentages): although present in the Kaggle feature set, they are derived from historical performance fragments and should be interpreted cautiously; any deployment should revisit feature definitions and monitoring if the pipeline is reused. **Critical assessment:** match prediction retains a **gambling-like nature**—like football or roulette, outcomes are partly stochastic. That is precisely why **sixty per cent accuracy is a respectable achievement** (chance is fifty per cent), and why literature-style benchmarks topping out near **sixty-five per cent** matter: they show how tight the practical ceiling is even under strong modelling choices (sometimes including neural networks). None of this is a licence for wagering; predictions are indicative, not guarantees (Section 1.5).

1.5 Ethical Considerations

1.5.1 Potential Biases

Temporal and concept drift. The dataset spans multiple competitive years. CSGO undergoes periodic balance patches, map pool rotations, and structural changes to competitive formats. A model trained on the full history therefore mixes regimes in which the mapping from pre-match statistics to outcomes may differ. That is classic *concept drift*: the joint distribution of features and labels shifts over time even if row-level features look well-formed.

Representation and coverage. Top teams appear in more featured matches and accumulate more rows, so the classifier is optimised for error on a distribution tilted toward elite play. Lower-tier teams, women's or regional scenes (if underrepresented in the source), and rare roster experiments may see higher error rates or miscalibrated probabilities. This is an equity-of-performance issue for any tool used to compare organisations fairly.

Ranking system and lag. World rankings aggregate past results with delay; a sudden roster upgrade or coaching change may not move ranks immediately. Because `rank_diff` is the strongest driver, systematic lag can produce confident wrong predictions after roster moves—exactly when human analysts also disagree.

Data collection and aggregation. International LANs and prominent online leagues dominate public databases; scrims, private practice, and psychological factors are absent. Team averages smooth away role structure (entry versus support) and IGL impact, so two teams with identical means can play differently in practice.

1.5.2 Mitigation Strategies

Operational mitigations. Use rolling time-based validation rather than a single random split when retraining; refresh models after major patches; add recency-weighted features (form over last N maps). When building evaluation sets, stratify or balance by tier and region so that accuracy reports do not hide poor performance on thin slices. Prefer relative constructs (`rank_diff`, differences in average rating) over raw absolute ranks where possible.

Governance and responsible use. Maintain interpretable baselines (logistic regression with documented coefficients or coefficients' signs) alongside black-box candidates so that failures can be diagnosed. Log predictions and outcomes in deployment for ongoing fairness-style audits across subgroups. Player-level statistics may be personal data in some jurisdictions; processing must respect consent, purpose limitation, and retention policies (GDPR-style principles).

Gambling and misuse. The model must **not** be used for betting or wagering. Outcomes retain large irreducible randomness (analogous to football or roulette). Readers should not mistake "only" sixty-one per cent for failure: in this setting it is **materially above chance** and in line with strong tabular results, while still **far** from a safe betting edge once margins, odds movement, and adaptation are considered (compare informal ceilings near sixty-five per cent on similar problems). Communicate uncertainty and abstain when probabilities are near one half.

Transparency. Publish the Kaggle provenance, cleaning rules, leakage removals, and the list of engineered features. Analysts and broadcasters can then decide when model output is supplementary to expert narrative versus when it should be ignored (e.g., novelty rosters).

1.5.3 Key Recommendations

1. Deploy **CatBoost** (or **LR with feature selection** when interpretability dominates) only as decision-support, not an autonomous decision system.
2. **Retrain** on sliding windows of recent matches; monitor post-patch degradation.
3. **Slice metrics** by team tier, region, and online versus LAN if sample sizes allow.
4. **Pair** outputs with human review for roster changes and format changes.
5. **Prohibit** betting and gambling use cases in product terms; document this restriction for end users.

Closing remarks (Dataset 1). Section 1 set out a reproducible workflow: leakage-aware cleaning, engineered team signals, exploratory analysis, supervised and unsupervised baselines, systematic hyperparameter search, and gradient boosting with CatBoost as the strongest test-set performer. The limitations above are not peripheral; they bound any legitimate use of the models. This section stands as a self-contained account of Dataset 1 for analysis and governance purposes.

Section 2 – Natural Language Processing and Deep Learning (Dataset 2)

Dataset 2 covers sentiment analysis of CSGO Steam reviews (positive vs. negative classification). Subsections 2.1–2.3 present preprocessing (including length and vocabulary diagnostics), neural model implementation with an architecture comparison on validation data, and evaluation using learning curves, a confusion matrix, classification metrics, and a decision-threshold sweep. Dataset source: https://www.kaggle.com/datasets/noahx1/csgo-steam-reviews.

Structured versus text tasks in one report. Section 1 and Section 2 answer different prediction questions on different modalities. Match outcome prediction uses dense numeric descriptors with a balanced label; the ceiling is **low in absolute terms but high relative to chance** because the task is **gambling-like**—headline accuracies in the low sixties are meaningful (fifty per cent random), and best-case results on comparable tasks are often quoted near **sixty-five per cent**. Sentiment classification uses sequential tokens with a skewed label distribution, where a model can achieve high accuracy while still failing the minority class unless metrics such as macro F1 are watched. Juxtaposing both problems illustrates a core data-science lesson: *good engineering and modern algorithms do not remove the need to choose metrics and priors that match the business question*. The Steam review task also raises distinct ethics (toxic language, review bombing, platform moderation) touched in Section 2.3.

2.1 Text Dataset Selection and Preprocessing

For this part of a project, we selected a publicly available dataset containing Steam reviews for the game Counter-Strike: Global Offensive (CS:GO) taken between 31st January 2023 to 1st March 2023. The dataset is available on Kaggle: https://www.kaggle.com/datasets/noahx1/csgo-steam-reviews.

It contains 15,640 user reviews, each labelled with a binary variable `voted_up`, where 1 represents a positive recommendation and 0 represents a negative recommendation. The dataset is highly imbalanced, with approximately 89% positive reviews and 11% negative reviews, which makes it suitable for a binary sentiment analysis task. Reviews are typically very short, with an average length of around five to six words and approximately 47 characters per review, reflecting the concise nature of many Steam user comments.

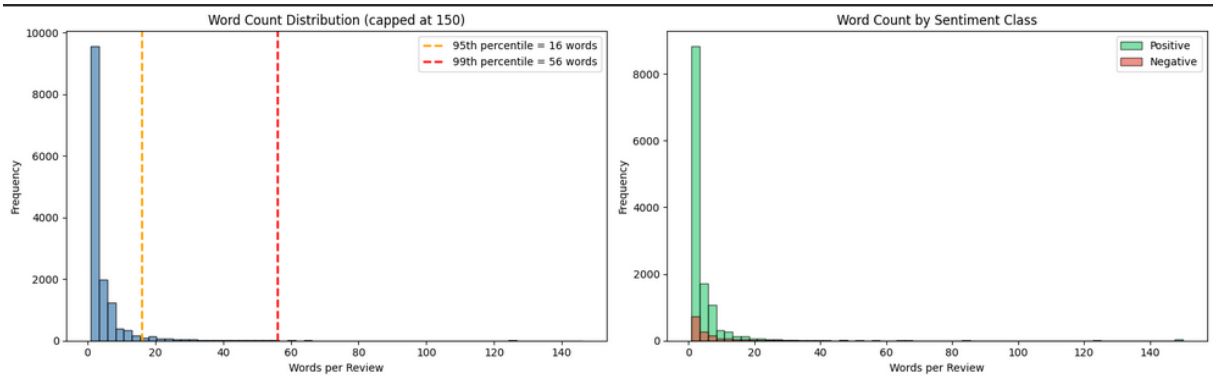

Figure 18: Word-count distribution for cleaned reviews and comparison by sentiment label (`voted_up`). Most reviews use very few tokens after cleaning; negative reviews are sparse but tend to appear across the same length range.

Figure 18 summarises length behaviour empirically: the bulk of reviews sit at low word counts, which supports the choice of a fifty-token sequence cap and explains why global pooling and short n-gram convolutions remain effective.

Before training the models, several preprocessing steps were applied to improve data quality. First, reviews with missing or empty text were removed. Reviews written mostly in non-Latin script, for example Cyrillic or ASCII art, were filtered out by computing a Latin character ratio and removing reviews with a ratio below 0.15. The remaining reviews were then cleaned by converting all text to lowercase, removing any URLs, HTML tags, punctuation, and non-alphabetic characters using regular expressions. The text was tokenized, and common English stop words were removed using the NLTK stop word list to reduce noise and improve feature quality. Reviews that became empty after cleaning were also discarded. The cleaned text was stored as a new feature and the sentiment label was converted into a binary target variable for model training.

For feature representation, pre-trained word embeddings from Global Vectors for Word Representation (GloVe) were used. Specifically, the GloVe 6B dataset with 100-dimensional vectors was downloaded from the Stanford NLP repository. Each word in the vocabulary was mapped to its corresponding embedding vector to create an embedding matrix, which was then used to initialise the embedding layer in the neural network models. Using pre-trained embeddings allows the model to leverage semantic relationships learned from large text corpora, improving generalization and enabling the model to capture contextual similarities between words.

Train, validation, and test discipline. After cleaning, the data are partitioned into training, validation, and held-out test sets. The validation split drives early stopping, learning-rate reduction, and hyperparameter selection; the test set is used only for final evaluation figures and tables. That separation mirrors Section 1's insistence on fitting

scalers on training data only: here, token frequencies and embedding alignment are computed from training corpora statistics so that validation and test reviews do not inflate vocabulary counts or distort rare-word handling.

Vocabulary and OOV behaviour. Limiting vocabulary to the twenty thousand most frequent tokens strikes a balance between coverage of CSGO slang (“toxic”, “smurfs”, map names) and matrix size. Out-of-vocabulary tokens map to a reserved index; random initialisation for missing GloVe rows lets fine-tuning recover domain usage during back-propagation. The Latin-character filter trades recall of multilingual community content for a more homogeneous token stream, which is an explicit scope limitation: non-English reviews are largely excluded.

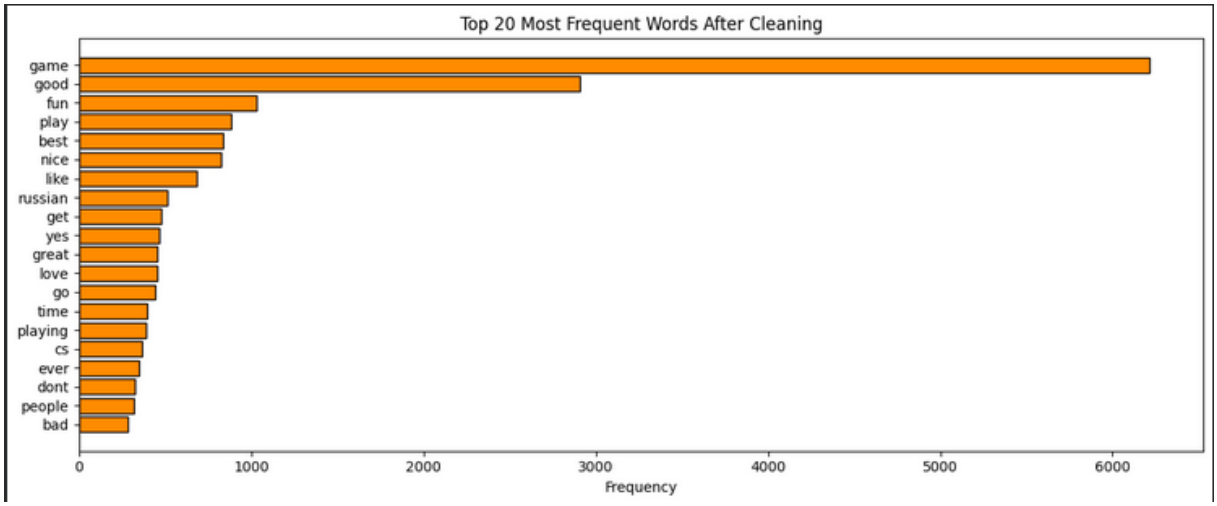

Figure 19: Twenty most frequent token types after cleaning and stop-word removal (training vocabulary statistics). Domain terms and short function words dominate; rare insults or map names appear lower in the tail and rely on the OOV bucket or sub-word generalisation.

Figure 19 confirms that the vocabulary head is interpretable: frequent items mix general English with CSGO-specific usage, so pre-trained embeddings plus fine-tuning are appropriate rather than training embeddings from scratch on this corpus alone.

2.2 Deep Learning Model Implementation

Following the preprocessing and feature representation, deep learning models were designed and trained to perform sentiment classification on the CS:GO Steam reviews dataset. The objective of the models is to predict whether a review expresses a positive or negative sentiment, based on the binary label. Text reviews represent sequential data; neural network architectures capable of modelling word order and contextual relationships were used.

To prepare the input for the neural networks, a custom vocabulary was constructed from the cleaned review corpus. The vocabulary size was limited to the 20,000 most frequent words, ensuring that the models focus on the informative tokens while keeping computational complexity manageable. Each review was converted into a sequence of integer indices representing the words in the vocabulary. Because neural networks require inputs of consistent length, the sequences were padded or truncated to a maximum length of 50 tokens, allowing the models to process all reviews in a fixed format.

For word representation, the pre-trained GloVe embeddings described in the previous section were used as the basis for the embedding layer. The embedding matrix was aligned with the custom vocabulary so that each token index corresponds to its respective 100-dimensional vector. Words not found in the pre-trained embedding set were assigned small random vectors so that they could still be learned during training. The embedding layer was configured with trainable parameters, allowing the neural network to fine-tune the vectors and adapt them to domain-specific language commonly found in gaming reviews.

To evaluate different approaches to modelling textual sentiment, three neural network architectures were implemented and compared: a Simple Recurrent Neural Network (RNN), a Long Short-Term Memory (LSTM) network, and a hybrid Convolutional Neural Network with Bidirectional LSTM (Conv1D + BiLSTM) model.

The Simple RNN model serves as a baseline architecture. It consists of an embedding layer followed by a SpatialDropout1D layer, a SimpleRNN layer with 128 units, a fully connected dense layer with ReLU activation, and a final output layer with a sigmoid activation function. The SpatialDropout1D layer randomly disables entire embedding dimensions during training, helping to regularise the embedding representation and reduce overfitting. The SimpleRNN layer processes the text sequentially and learns patterns based on word order. However, traditional RNNs can struggle to retain information from earlier parts of a sequence due to the vanishing gradient problem.

To address this limitation, a second architecture replaces the SimpleRNN layer with an LSTM layer. LSTMs introduce internal gating mechanisms that regulate the flow of information through the network. These gates allow the model to retain important contextual information across longer sequences, making LSTMs particularly suitable for natural language processing tasks. In this architecture, the LSTM layer is followed by a dense layer with ReLU activation, a Dropout layer for regularisation, and a final sigmoid output layer that predicts the probability of a positive review.

We decided to include a more advanced architecture combining convolutional and recurrent neural network techniques. This model consists of an embedding layer,

SpatialDropout1D regularisation, a Conv1D layer with 128 filters and a kernel size of three, a Bidirectional LSTM layer, GlobalMaxPooling1D, and several fully connected layers. The convolutional layer acts as a feature extractor by identifying local n-gram patterns in the text, such as short phrases that strongly suggest sentiment. The Bidirectional LSTM processes the review sequence in both forward and backward directions, allowing the model to capture contextual information from both preceding and following words in the sequence. This helps the network understand more complex sentiment expressions.

The output of the Bidirectional LSTM is passed to a GlobalMaxPooling1D layer, which extracts the strongest feature signals across the sequence and converts the variable-length sequence representation into a fixed-length feature vector. This representation is then processed by a dense layer with ReLU activation, followed by Batch Normalization, which stabilises and accelerates training by normalising intermediate activations. A Dropout layer is also applied to reduce overfitting by randomly deactivating neurons during training. Finally, a single neuron with a sigmoid activation function produces the probability that the review expresses positive sentiment.

Validation accuracy across architectures. After training each architecture with the same preprocessing, tokenisation, and class-weighting scheme, validation-set accuracy and per-metric breakdowns were compared before committing to a full grid search on the strongest design. Figure 20 summarises the pattern: the Conv1D + BiLSTM stack improves on the plain RNN and LSTM baselines, matching the qualitative expectation that local n-gram features and bidirectional context help on short, noisy Steam text.

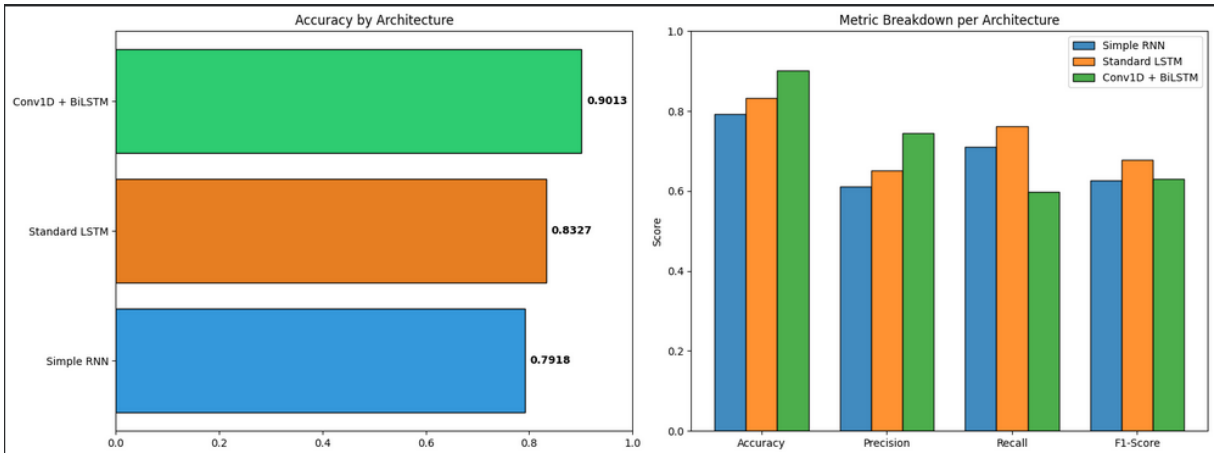

Figure 20: Accuracy by architecture with a per-metric view (validation set). Used to justify focusing hyperparameter search on the Conv1D + BiLSTM model; final test metrics are reported separately in Section 2.3.

All models were trained using the binary cross-entropy loss function, which is appropriate for binary classification tasks. The Adam optimiser was used due to its efficiency

and adaptive learning rate capabilities. Since the dataset is highly imbalanced, with a significantly larger number of positive reviews than negative ones, we decided to implement a class weighting during training to ensure that minority class samples were given appropriate importance.

Loss and class weights (conceptual). Binary cross-entropy for label $y \in \{0, 1\}$ and predicted probability $\hat{p}$ is $-\big(y \log \hat{p} + (1-y) \log(1-\hat{p})\big)$, averaged over mini-batches. Class weights rescale contributions so that a misclassified rare negative review contributes similarly to total gradient magnitude as several easy positives, preventing collapse to the trivial always-positive predictor. This is complementary to macro F1 evaluation: weights shape training; F1 shapes how we judge success.

To improve training stability and prevent overfitting, early stopping was implemented. This technique monitors the validation loss during training and stops the training process if the model's performance stops improving for several epochs. In addition, the final model incorporated a learning rate scheduling strategy using ReduceLROnPlateau, which automatically reduces the learning rate when validation performance plateaus. This allows the model to make finer adjustments to its weights during later training stages.

In order to further improve model performance, several hyperparameters were automatically tuned. A grid search approach was used to explore different combinations of batch size, dropout rate, and learning rate. Specifically, batch sizes of 64, 128, and 256, dropout rates of 0.1, 0.2, and 0.3, and learning rates of 0.001, 0.0005, and 0.0001 were evaluated. Each combination of parameters was tested using the Conv1D + Bidirectional LSTM architecture, which showed the strongest initial performance.

For each hyperparameter configuration, the model was trained for several epochs and evaluated using multiple performance metrics, including accuracy, precision, recall, and macro F1-score. Because the dataset is imbalanced, the macro F1-score was used as the primary metric, as it provides a balanced measure of performance across both classes. The grid search procedure recorded the results for each configuration and selected the parameter combination that achieved the highest macro F1-score.

The final model therefore uses the Conv1D + Bidirectional LSTM architecture with the optimal hyperparameters identified through grid search. This model integrates pre-trained word embeddings, convolutional feature extraction, bidirectional sequence modelling, and regularisation techniques such as dropout and batch normalization. Through this combination of architectural design and hyperparameter optimisation, the model is able to effectively capture both local sentiment patterns and broader contextual relationships within the review text.

Reproducibility and compute. Neural training is stochastic (mini-batch order, dropout masks, weight initialisation). Random seeds for NumPy and TensorFlow are fixed where applicable for reproducibility. Training time scales with sequence length (fifty tokens), vocabulary cap, and batch size; grid search multiplies wall-clock cost, which is why the search is restricted to the strongest architecture rather than repeating the same grid on weaker RNN baselines.

2.3 Evaluation and Insights

To assess the final sentiment classification model, we use accuracy, precision, recall, and F1 (including macro F1 under imbalance), supported by learning curves, a confusion matrix, a classification-report summary, and a decision-threshold sweep. Together these give a view of fit, generalisation, and class balance.

Headline test-set scores (held-out split). The final Conv1D + BiLSTM checkpoint achieves **88%** overall accuracy on two thousand eight hundred and eighty-seven test reviews. **F1-scores** (the harmonic mean of precision and recall for each class) are: **negative class F1 = 0.42** (precision 0.42, recall 0.43); **positive class F1 = 0.93** (precision 0.93, recall 0.93); **macro-averaged F1 = 0.68**; **weighted-averaged F1 = 0.88**, matching the weighted precision and recall in Figure 23. Macro F1 is far below headline accuracy because the negative class is rare; we therefore treat macro F1 as the primary scalar summary for imbalance, alongside per-class F1.

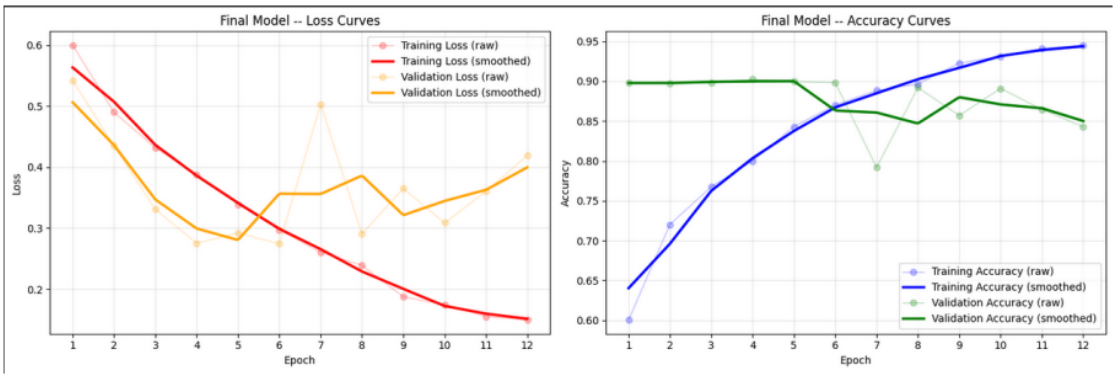

Figure 21: Training and validation loss and accuracy curves.

Figure 21 shows training and validation loss and accuracy over epochs. These plots allow direct observation of how the model learns during training. The loss curves demonstrate whether the model is successfully minimising prediction error over time, while the accuracy curves provide insight into how well the model performs on both training and validation datasets. To make the trends easier to interpret, the curves are smoothed while still preserving the raw epoch values. The plots indicate that the model converges steadily, with training and validation metrics following similar trajectories. This

suggests that the combination of dropout regularisation, batch normalisation, and early stopping helps prevent severe overfitting while allowing the model to learn meaningful patterns from the data.

Confusion matrix analysis

To better understand how the model performs across classes, a confusion matrix was generated for the final predictions.

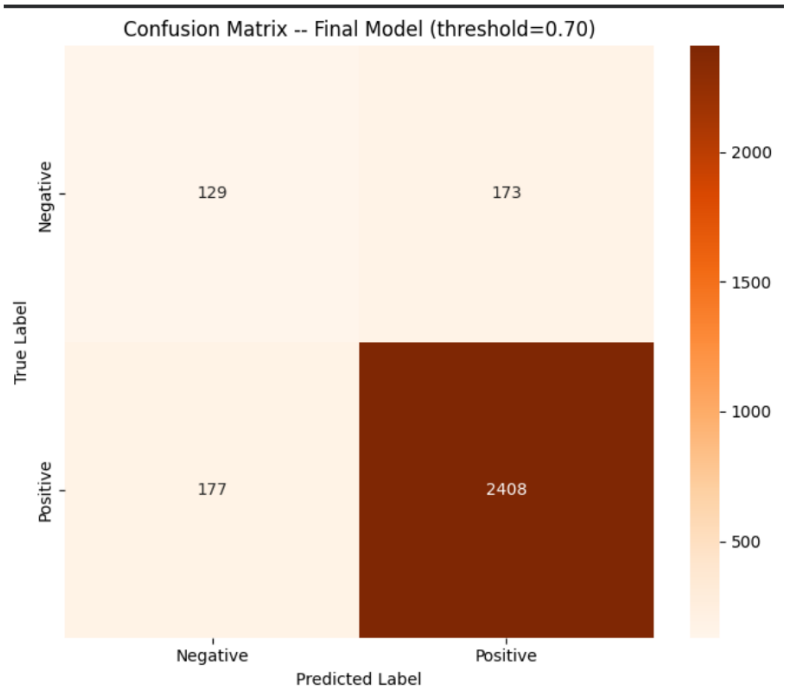

Figure 22: Confusion matrix for final sentiment predictions.

Figure 22 visualises true negatives, false positives, false negatives, and true positives. On the test split, the counts are TN = 129, FP = 173, FN = 177, TP = 2,408 (total 2,887), so most mass lies on true positives as expected under imbalance. This representation provides a clear view of how well the model distinguishes between positive and negative reviews. Because the dataset is highly imbalanced (approximately 89% positive and 11% negative reviews), examining the confusion matrix is particularly important to ensure that the model does not simply predict the majority class. The heatmap shows that the model correctly identifies a large number of positive reviews while also maintaining reasonable performance on the minority negative class. Although some misclassifications remain, particularly among very short or ambiguous reviews, the confusion matrix demonstrates that the model is capable of learning meaningful sentiment patterns despite the imbalance.

Classification metrics

In addition to the confusion matrix, the final model was evaluated using several standard classification metrics.

CLASSIFICATION REPORT (Final Model, threshold=0.70)

	precision	recall	f1-score	support
Negative	0.42	0.43	0.42	302
Positive	0.93	0.93	0.93	2585
accuracy			0.88	2887
macro avg	0.68	0.68	0.68	2887
weighted avg	0.88	0.88	0.88	2887

Figure 23: Final classification report summary.

Figure 23 tabulates precision, recall, and F1-score for each class, plus accuracy and the macro and weighted averages. The same numbers in prose: **accuracy 0.88**; negative-class **precision 0.42**, **recall 0.43**, **F1 0.42**; positive-class **precision 0.93**, **recall 0.93**, **F1 0.93**; **macro F1 0.68** (macro precision and recall 0.68); **weighted F1 0.88** (weighted precision and recall 0.88). Weighted averages track overall accuracy because support is skewed; macro F1 stays low until the rare class is handled well. While overall accuracy provides a useful measure of general predictive performance, it can be misleading in imbalanced datasets. For this reason, macro F1 is emphasised alongside per-class F1. Macro F1 treats each class equally by averaging class F1-scores, making it a more reliable headline than accuracy alone when class distributions are uneven.

Decision threshold optimisation

Although the default probability threshold is typically 0.5, we also sweep thresholds to confirm whether that operating point is appropriate.

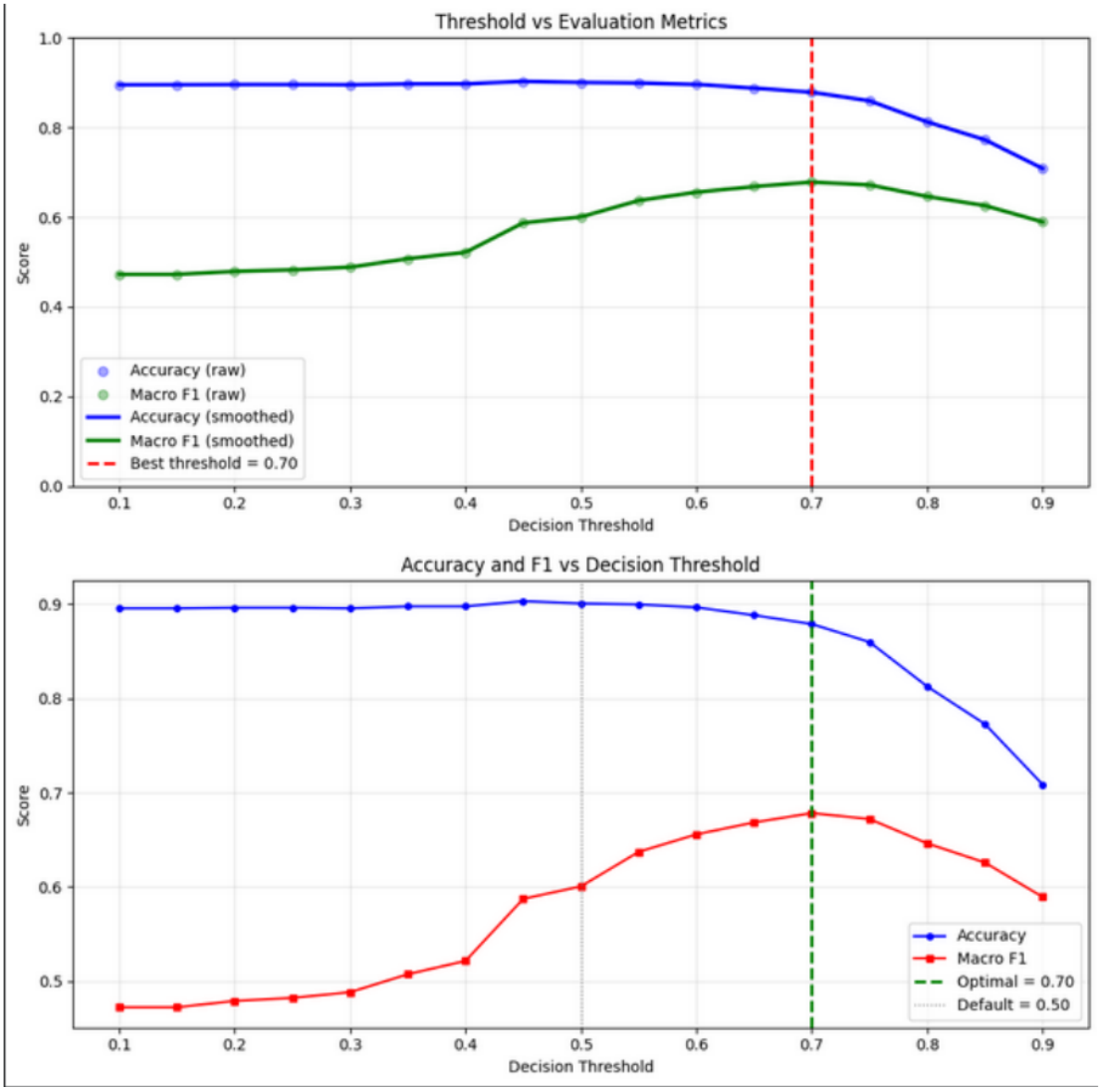

Figure 24: Decision threshold sweep: accuracy and macro F1 across thresholds.

Figure 24 shows the threshold sweep. The decision threshold was varied from 0.10 to 0.90, and both accuracy and macro F1 were recorded at each value. At the **default 0.50** cut-off, accuracy is higher (about **90%**) but **macro F1 is only about 0.60**—the model favours the majority class. The sweep identifies a **best threshold near 0.70**, where **macro F1 peaks at about 0.68** with **accuracy about 88%**—a deliberate trade-off: we sacrifice a few points of headline accuracy to lift F1 on the minority class. Final reported metrics and figures (Figures 22–23) use this operating point unless noted otherwise.

Strengths of the approach

The evaluation highlights several strengths of the implemented sentiment classification system. First, the custom preprocessing pipeline provides full control over the data cleaning and tokenisation process, ensuring that noisy or irrelevant content is removed before training. This improves the quality of the textual input provided to the model.

Second, the comparison of multiple neural architectures allows the most effective model design to be identified. Among the tested models, the Conv1D + Bidirectional LSTM architecture demonstrates superior performance by combining local pattern extraction with contextual sequence modelling. Third, the use of grid search hyperparameter tuning and class weighting helps mitigate the effects of the strong class imbalance in the dataset. By adjusting batch size, learning rate, and dropout rate, the training process identifies parameter combinations that maximise macro F1-score; the resulting **test macro F1 is 0.68** with **class F1 scores 0.42** (negative) and **0.93** (positive), as tabulated in Figure 23. Another advantage is the use of trainable GloVe embeddings, which allow the model to refine pre-trained word representations and adapt them to domain-specific language commonly found in gaming reviews. Finally, the evaluation process itself is comprehensive. By combining learning curves, confusion matrices, classification metrics, and threshold analysis, we obtain clear visual confirmation of model behaviour, as illustrated in Figures 21, 22, 23, and 24.

Limitations

Despite the strong performance of the model, several limitations remain. Sentiment classification models based on recurrent networks may struggle to interpret sarcasm, irony, or complex linguistic structures, which are common in informal online reviews. For example, phrases that appear positive in isolation may actually convey negative sentiment depending on context. In addition, the dataset contains very short reviews and potentially noisy labels, which can limit the maximum achievable performance. Extremely brief comments may lack sufficient context for accurate sentiment prediction, even when advanced neural architectures are used.

Platform dynamics and label semantics. Steam's `voted_up` flag reflects whether the user recommends the game, not a fine-grained sentiment score. A negative recommendation can mean disapproval of recent patches, anti-cheat friction, or toxicity in matchmaking—topics that overlap only partially with the literal text of a five-word review. Review waves tied to major updates ("review bombing") can shift the joint distribution of text and labels without the language model seeing the external event, hurting generalisation across time. Community norms also evolve: memes and slang that read as negative to a human may lack negative training examples.

Ethical deployment. Automated sentiment or moderation systems trained on public reviews can entrench majority opinions and mis-handle non-English or coded language unless explicitly tested. Any production use should keep humans in the loop for appeals, publish error rates by demographic or regional slice where feasible, and avoid using model scores as the sole reason to shadow-ban or sanction accounts.

Future improvements

Future work could explore the use of transformer-based models, such as Bidirectional Encoder Representations from Transformers (BERT). Transformer architectures use self-attention mechanisms to capture relationships between words across an entire sentence simultaneously. Pre-trained transformer models also incorporate rich contextual embeddings learned from very large corpora, which could significantly improve performance on nuanced sentiment expressions and sarcasm. Integrating such models could further enhance the system's ability to understand complex review language and achieve higher classification accuracy.

Summary

Overall, the evaluation demonstrates that the developed sentiment classification system performs effectively on the CS:GO Steam reviews dataset: **88%** test accuracy, **macro F1 of 0.68**, **weighted F1 of 0.88**, and class F1 scores **0.42** (negative) and **0.93** (positive) on the held-out set—the spread between macro and weighted F1 illustrates the skewed class distribution. Standard practice is followed throughout: learning curves (Figure 21), a confusion matrix (Figure 22), per-class classification metrics (Figure 23), and threshold optimisation (Figure 24). These techniques provide quantitative and visual insight into model performance and justify the chosen architecture and hyperparameters. Together, they confirm that the final Conv1D + Bidirectional LSTM model captures meaningful sentiment patterns in the review text; balanced performance across classes remains limited by the rarity of negative reviews despite class weighting.

Synthesis across Dataset 1 and Dataset 2

Both parts of this coursework exercise the same scientific habits under different data regimes. Dataset 1 foregrounds *leakage control*, *calibrated expectations* about **gambling-like** noisy targets, and *model choice under a tabular ceiling*: gradient-boosted trees (CatBoost) edge out logistic regression with feature selection. Absolute accuracy sits near **sixty-one per cent—good** against a fifty per cent baseline and close to informal **sixty-five per cent** best-case benchmarks reported on similar match models (including neural-network approaches in some studies)—showing that better algorithms help but cannot remove inherent match randomness. Dataset 2 foregrounds *representation learning* (embeddings, convolutions, recurrence), *class imbalance*, and *richer evaluation dashboards* (macro F1 $\approx$ 0.68 vs. weighted F1 $\approx$ 0.88 on Steam reviews, threshold sweeps) because headline accuracy alone would mask collapse on negative reviews.

Methodologically, both streams rely on held-out evaluation, explicit handling of missing or

dirty inputs, and ethical reflection on who is represented in the data and how predictions might be misused. Practitioners should not export the match predictor into high-stakes wagering contexts, and should not export the sentiment model into fully automated moderation without human oversight, especially given sarcasm and label noise. Future extensions might unify the themes by building multi-input models (tabular team stats plus community sentiment features) while retaining the governance lessons from Sections 1.5 and 2.3.

Checklist for replication. A reader who wishes to reproduce our conclusions should: (one) start from the same Kaggle exports and verify row counts after cleaning; (two) confirm removal of `t1_points` and `t2_points`, exclusion of draws, and the target row counts (three thousand seven hundred and sixty-three) before training; (three) refit `StandardScaler` on train only; (four) reuse the stratified eighty/twenty split seed if exact duplication is required; (five) for NLP, rebuild vocabulary from train reviews only and re-download matching GloVe files; (six) compare final neural-network metrics on the held-out test set to the values reported in Section 2.3. Small numerical drifts across machines are normal for deep learning but should not change qualitative conclusions if seeds and library versions align.

What we would do with more time. For structured match data, hierarchical models that partially pool by team ID could exploit repeated matchups but require careful regularisation to avoid overfitting small rosters. Map-specific submodels would need sufficient samples per map. For text, transformer fine-tuning (e.g., small BERT or DistilBERT) with class-weighted loss would be the next accuracy-focused step, at higher GPU cost. For both problems, deployment-oriented work would emphasise calibration (Platt scaling or isotonic regression on validation outputs) and continuous monitoring dashboards rather than marginal gains in offline accuracy.

References

Chen, T. and Guestrin, C. (2016) 'XGBoost: A Scalable Tree Boosting System', *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794. Available at: https://doi.org/10.1145/2939672.2939785 (Accessed: 7 February 2025).

Harris, C.R., Millman, K.J., van der Walt, S.J. *et al.* (2020) 'Array programming with NumPy', *Nature*, 585, pp. 357–362. Available at: https://doi.org/10.1038/s41586-020-2649-2 (Accessed: 7 February 2025).

HLTV.org (2024) *CSGO Match Statistics*. Available at: https://www.hltv.org (Accessed: 7 February 2025).

Kaggle (2024) *Counter-Strike: Global Offensive matches*. Available at: https://www.kaggle.com/datasets/gabrieltardochi/counter-strike-global-offensive-matches (Accessed: 7 February 2025).

Kaggle (2024) *CSGO Steam Reviews*. Available at: https://www.kaggle.com/datasets/noahx1/csgo-steam-reviews (Accessed: 7 February 2025).

McKinney, W. (2010) 'Data structures for statistical computing in Python', in van der Walt, S. and Millman, J. (eds) *Proceedings of the 9th Python in Science Conference*, pp. 56–61.

Pedregosa, F., Varoquaux, G., Gramfort, A. *et al.* (2011) 'Scikit-learn: Machine learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830. Available at: https://jmlr.org/papers/v12/pedregosa11a.html (Accessed: 7 February 2025).